

Generic LinkedList<T> Collection Class in C# with Examples

In this article, I am going to discuss **Generic LinkedList<T> Collection Class in C#** with Examples. Please read our previous article where we discussed [Generic SortedDictionary<TKey, TValue> Collection Class in C#](#) with Examples. At the end of this article, you will understand the following pointers with examples.

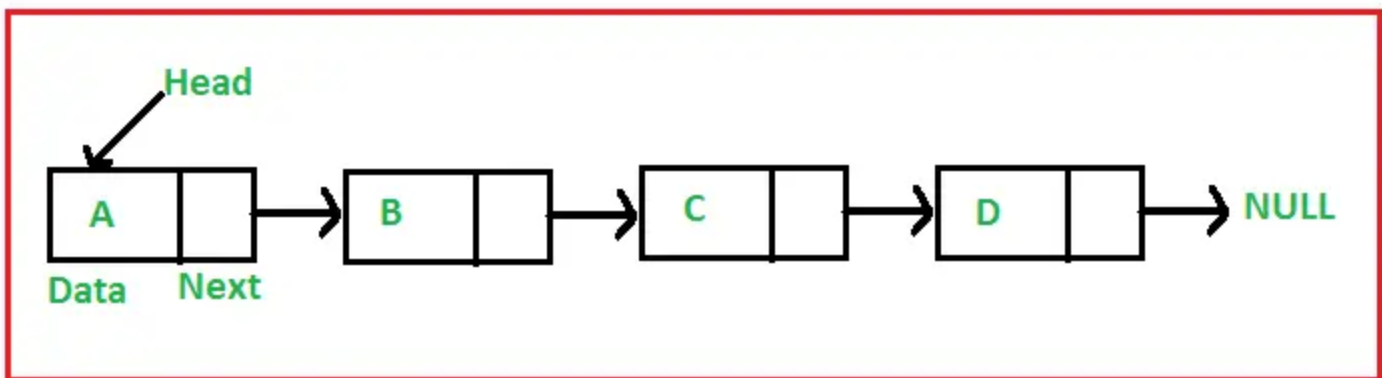
1. [What is LinkedList<T> in C#?](#)
2. [How to Create a LinkedList<T> Collection in C#?](#)
3. [How to Add Elements into a Generic LinkedList<T> Collection in C#?](#)
4. [How to Access a Generic LinkedList<T> Collection in C#?](#)
5. [How to Remove Elements from the LinkedList<T> collection in C#?](#)
6. [How to check the Availability of elements in a Generic LinkedList<T> Collection in C#?](#)
7. [How to Add a Node at the First Position of a Linked List in C#?](#)
8. [How to Add a Node at the Last Position of a Linked List in C#?](#)
9. [How to Add a Node After a Given Node of a Linked List in C#?](#)
10. [How to Add a Node Before a Given Node of a Linked List in C#?](#)
11. [Generic LinkedList<T> Collection with Complex Type in C#](#)
12. [Advantages of Linked List in C#](#)

What is LinkedList<T> in C#?

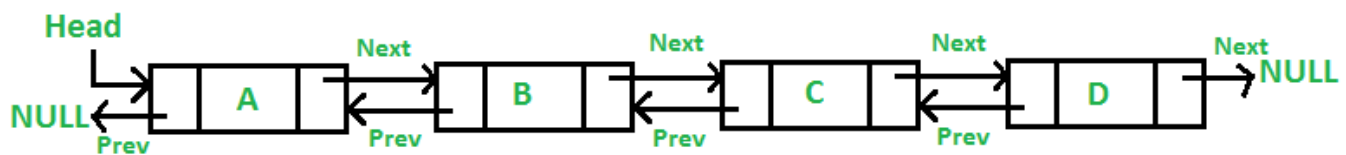
A LinkedList is a linear data structure used for storing the elements in a non-contiguous manner. The elements in a linked list are linked with each other using pointers. Or in other words, LinkedList consists of nodes where each node contains a data field and a reference(link) to the next node in the list. So, each node contains two parts.

1. **Data** – Each node of a linked list can store data.
2. **Address** – Each node of a linked list contains an address to the next node, called “Next”.

In simple words, a Linked List consists of nodes where each node contains a data field and a reference (link or pointer) to the next node in the list. The following diagram shows a Single Linked List.



The LinkedList<T> is a Generic Collection Class in C# which implements a double-ended linked list and is used to store a collection of the same type of values. As it is a doubly linked list, therefore, each node points forward to the Next node and backward to the Previous node. A Doubly Linked List (DLL) contains an extra pointer, typically called the previous pointer, together with the next pointer and data as shown in the below image.



Note: In C#, the `LinkedList<T>` is a dynamic collection that grows, according to the need of your program. It also provides fast insertion and removing elements.

Points to Remember while working with Linked List:

1. The `LinkedList<T>` belongs to the `System.Collections.Generic` namespace and implements `ICollection`, `ICollection<T>`, `IEnumerable`, `IEnumerable<T>`, `ISerializable`, and `IDeserializationCallback` interfaces.
2. `LinkedList<T>` is a general-purpose linked list. It supports enumerators.
3. You can remove nodes and reinsert them, either in the same list or in another list, which results in no additional objects allocated on the heap. Insertion and removal are $O(1)$ operations.
4. Each node in a `LinkedList<T>` object is of the type `LinkedListNode<T>`.
5. The `LinkedList` class does not support chaining, splitting, cycles, or other features that can leave the list in an inconsistent state.
6. If the `LinkedList` is empty, the `First` and `Last` properties contain null.
7. The `LinkedList` is doubly linked, therefore, each node points forward to the `Next` node and backward to the previous node.
8. The capacity of a `LinkedList` is the number of elements the `LinkedList` can hold.
9. In `LinkedList`, it is allowed to store duplicate elements of the same type.

How to Create a `LinkedList<T>` Collection in C#?

The Generic `LinkedList<T>` Collection class in C# provided the following Constructors that we can use to create an instance of the `LinkedList<T>` class.

1. **`LinkedList()`:** It initializes a new instance of the Generic `LinkedList` class that is empty.
2. **`LinkedList(IEnumerable<T> collection)`:** It initializes a new instance of the Generic `LinkedList` class that contains elements copied from the specified `System.Collections.IEnumerable` and has sufficient capacity to accommodate the number of elements copied. Here, the parameter `collection` specifies the `System.Collections.IEnumerable` whose elements are copied to the new Generic `LinkedList`. If the collection is null, then it will throw `ArgumentNullException`.
3. **`LinkedList(SerializationInfo info, StreamingContext context)`:** It initializes a new instance of the Generic `LinkedList` class that is serializable with the specified `System.Runtime.Serialization.SerializationInfo` and `System.Runtime.Serialization.StreamingContext`. Here, the parameter `info` specifies a `System.Runtime.Serialization.SerializationInfo` object containing the information required to serialize the Generic `LinkedList`. The parameter `context` specifies a `System.Runtime.Serialization.StreamingContext` object containing the source and destination of the serialized stream associated with the Generic `LinkedList`.

Let us see how to create a `LinkedList<T>` collection using the `LinkedList()` constructor in C#:

Step1:

As the Generic LinkedList<T> Collection Class belongs to System.Collections.Generic namespace, so first, we need to import the System.Collections.Generic namespace into our program as follows:

```
using System.Collections.Generic;
```

Step2:

Next, we need to create an instance of the LinkedList<T> collection class using the LinkedList() constructor as follows:

```
LinkedList<Type_of_linkedlist> linkedlist_name = new LinkedList<Type_of_linkedlist>();
```

How to Add Elements into a Generic LinkedList<T> Collection in C#?

If you want to add elements to a Generic LinkedList Collection, then you need to use the following methods provided by the Generic LinkedList<T> class as per your requirements.

1. **AddAfter(LinkedListNode<T> node, LinkedListNode<T> newNode):** This is used to add the specified new node after the specified existing node in the Generic LinkedList.
2. **AddAfter(LinkedListNode<T> node, T value):** This is used to add a new node containing the specified value after the specified existing node in the Generic LinkedList.
3. **AddBefore(LinkedListNode<T> node, LinkedListNode<T> newNode):** This method is used to add the specified new node before the specified existing node in the Generic LinkedList.
4. **AddBefore(LinkedListNode<T> node, T value):** This method is used to add a new node containing the specified value before the specified existing node in the Generic LinkedList.
5. **AddFirst(LinkedListNode<T> node):** This is used to add the specified new node at the start of the Generic LinkedList.
6. **AddFirst(T value):** This is used to add a new node containing the specified value at the start of the Generic LinkedList.
7. **AddLast(LinkedListNode<T> node):** This is used to add the specified new node at the end of the Generic LinkedList.
8. **LinkedListNode<T> AddLast(T value):** This is used to add a new node containing the specified value at the end of the Generic LinkedList.

For example, here we are creating a generic LinkedList<T> collection by specifying the type as a string as follows and then adding the elements using the AddLast method.

```
LinkedList<string> linkedList = new LinkedList<string>();  
linkedList.AddLast("India");  
linkedList.AddLast("USA");  
linkedList.AddLast("Srilanka");
```

Note: As the LinkedList<T> collection class does not have Add method which takes the T parameter, so we cannot use the Collection Initializer syntax to add elements while creating the collection.

How to Access a Generic LinkedList<T> Collection in C#?

You can access the elements of a Generic LinkedList<T> Collection in C# using For Each loop as follows:

```
foreach (var item in linkedList)  
{  
    Console.WriteLine(item);  
}
```

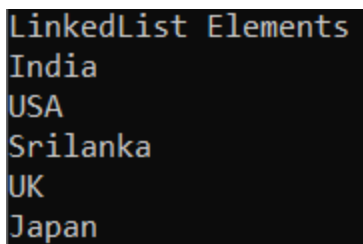
Note: As the `LinkedList<T>` Collection class does not have any indexer, so we cannot access elements using Indexes, and hence we cannot also access the elements using For loop.

Example to Understand How to Create a Generic `LinkedList<T>` Collection and Add Elements in C#:

For a better understanding of how to create a Generic `LinkedList<T>` Collection and how to add elements to the collection, and how to access the elements from the collection, please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericLinkedListCollection
{
    public class Program
    {
        public static void Main()
        {
            LinkedList<string> linkedList = new LinkedList<string>();
            linkedList.AddLast("India");
            linkedList.AddLast("USA");
            linkedList.AddLast("Srilanka");
            linkedList.AddLast("UK");
            linkedList.AddLast("Japan");
            Console.WriteLine("LinkedList Elements");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:



```
LinkedList Elements
India
USA
Srilanka
UK
Japan
```

How to Remove Elements From `LinkedList<T>` Collection in C#?

The Generic `LinkedList<T>` Collection Class in C# provides the following methods to remove elements from the `LinkedList` collection.

1. **Remove(LinkedListNode<T> node):** The Remove(LinkedListNode<T> node) method is used to remove the specified node from the Generic LinkedList.
2. **Remove(T value):** The Remove(T value) method is used to remove the first occurrence of the specified value from the Generic LinkedList.
3. **RemoveFirst():** The RemoveFirst() method is used to remove the node at the start of the Generic Linked List.
4. **RemoveLast():** The RemoveLast() method is used to remove the node at the end of the Generic Linked List.
5. **Clear():** The Clear() method is used to remove all nodes from the Generic LinkedList.

Let us see an example to understand the above methods of Generic LinkedList<T> Collection Class in C#. Please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericLinkedListCollection
{
    public class Program
    {
        public static void Main()
        {
            LinkedList<string> linkedList = new LinkedList<string>();
            linkedList.AddLast("India");
            LinkedListNode<string> USANode = linkedList.AddLast("USA");
            linkedList.AddLast("Srilanka");
            LinkedListNode<string> UKNode = linkedList.AddLast("UK");
            linkedList.AddLast("Japan");
            Console.WriteLine("Initial LinkedList Elements");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            // Removing Element using Remove(LinkedListNode) method
            // linkedList.Remove(linkedList.First); //Remove First Node
            // linkedList.Remove(linkedList.Last); //Remove Second Node
            linkedList.Remove(USANode); //Remove Specific Node
            Console.WriteLine("\nLinkedList Elements After Remove(USANode)");
            foreach (string item in linkedList)
            {
                Console.WriteLine(item);
            }
            // Removing Element using Remove(T) method
            linkedList.Remove("UK");
            Console.WriteLine("\nLinkedList Elements After Remove(UK)");
            foreach (string item in linkedList)
            {
                Console.WriteLine(item);
            }
            // Removing Element using RemoveFirst() method
            linkedList.RemoveFirst();
            Console.WriteLine("\nLinkedList Elements After RemoveFirst()");
```

```

        foreach (string item in linkedList)
        {
            Console.WriteLine(item);
        }
        // Removing Element using RemoveLast() method
        linkedList.RemoveLast();
        Console.WriteLine("\nLinkedList Elements After RemoveLast()");
        foreach (string item in linkedList)
        {
            Console.WriteLine(item);
        }
        // Removing Element using Clear() method
        linkedList.Clear();
        Console.WriteLine($"\\nLinkedList Count After Clear(): {linkedList.Count}");
        Console.ReadKey();
    }
}

```

Output:

```

Initial LinkedList Elements
India
USA
Srilanka
UK
Japan

LinkedList Elements After Remove(USANode)
India
Srilanka
UK
Japan

LinkedList Elements After Remove(UK)
India
Srilanka
Japan

LinkedList Elements After RemoveFirst()
Srilanka
Japan

LinkedList Elements After RemoveLast()
Srilanka

LinkedList Count After Clear(): 0

```

How to check the Availability of elements in a Generic LinkedList<T> Collection in C#?

If you want to check whether an element exists or not in the Generic LinkedList<T> Collection in C#, then you can use the following Contains(T value) method provided by the Generic LinkedList<T> class.

1. **Contains(T value):** This method is used to determine whether a value is in the Generic LinkedList. Here, the parameter value specifies the value to locate in the Generic LinkedList. The value can be null for reference types. It returns true if the value is found in the Generic LinkedList; otherwise, false.

Let us understand this with an example. The following example shows how to use the both Contains(T value) method of the Generic LinkedList collection class in C#.

```
using System;
using System.Collections.Generic;
namespace GenericLinkedListCollection
{
    public class Program
    {
        public static void Main()
        {
            LinkedList<string> linkedList = new LinkedList<string>();
            linkedList.AddLast("India");
            linkedList.AddLast("USA");
            linkedList.AddLast("Srilanka");
            linkedList.AddLast("UK");
            linkedList.AddLast("Japan");
            Console.WriteLine("LinkedList Elements");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            //Checking the value using the ContainsValue method
            Console.WriteLine("\nIs India value Exists : " +
linkedList.Contains("India"));
            Console.WriteLine("\nIs Bangladesh value Exists : " +
linkedList.Contains("Bangladesh"));
            Console.ReadKey();
        }
    }
}
```

Output:

```
LinkedList Elements
India
USA
Srilanka
UK
Japan

Is India value Exists : True

Is Bangladesh value Exists : False
```

Linked List Operations in C#

How to Add a Node at the First Position of a Linked List in C#?

If you want to add a node at the first position of a linked list, then you need to use the `AddFirst()` method of the `Generic LinkedList<T>` class. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericLinkedListCollection
{
    public class Program
    {
        public static void Main()
        {
            LinkedList<string> linkedList = new LinkedList<string>();
            linkedList.AddLast("India");
            linkedList.AddLast("USA");
            linkedList.AddLast("Srilanka");

            Console.WriteLine("LinkedList Elements");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.WriteLine("\nAfter Adding a Node at First Position");
            linkedList.AddFirst("UK");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
LinkedList Elements
India
USA
Srilanka

After Adding a Node at First Position
UK
India
USA
Srilanka
```

How to Add a Node at the Last Position of a Linked List in C#?

If you want to add a node at the last position of a linked list, then you need to use the AddLast() method of the Generic LinkedList<T> class. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericLinkedListCollection
{
    public class Program
    {
        public static void Main()
        {
            LinkedList<string> linkedList = new LinkedList<string>();
            linkedList.AddLast("India");
            linkedList.AddLast("USA");
            linkedList.AddLast("Srilanka");

            Console.WriteLine("LinkedList Elements");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.WriteLine("\nAfter Adding a Node at Last Position");
            linkedList.AddLast("UK");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
LinkedList Elements
India
USA
Srilanka

After Adding a Node at Last Position
India
USA
Srilanka
UK
```

How to Add a Node After a Given Node of a Linked List in C#?

If you want to add a node after a given node of a linked list, then you need to use the `AddAfter()` method of the `Generic LinkedList<T>` class. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericLinkedListCollection
{
    public class Program
    {
        public static void Main()
        {
            LinkedList<string> linkedList = new LinkedList<string>();
            linkedList.AddLast("India");
            LinkedListNode<string> USANode = linkedList.AddLast("USA");
            linkedList.AddLast("Srilanka");

            Console.WriteLine("LinkedList Elements");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.WriteLine("\nAfter Adding a Node After USA Node");
            linkedList.AddAfter(USANode, "UK");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
LinkedList Elements
India
USA
Srilanka

After Adding a Node After USA Node
India
USA
UK
Srilanka
```

How to Add a Node Before a Given Node of a Linked List in C#?

If you want to add a node before a given node of a linked list, then you need to use the `AddBefore()` method of the Generic `LinkedList<T>` class. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericLinkedListCollection
{
    public class Program
    {
        public static void Main()
        {
            LinkedList<string> linkedList = new LinkedList<string>();
            linkedList.AddLast("India");
            LinkedListNode<string> USANode = linkedList.AddLast("USA");
            linkedList.AddLast("Srilanka");

            Console.WriteLine("LinkedList Elements");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.WriteLine("\nAfter Adding a Node Before USA Node");
            linkedList.AddBefore(USANode, "UK");
            foreach (var item in linkedList)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
LinkedList Elements
India
USA
Srilanka

After Adding a Node Before USA Node
India
UK
USA
Srilanka
```

Generic LinkedList<T> Collection with Complex Type in C#:

As of now, we have used the built-in data types such as int, string, etc with LinkedList Class. Now, let us see how to create a Generic LinkedList collection using Complex types. Let us create a class called Student and then let us create a LinkedList collection of Student types as shown in the below example.

```
using System;
using System.Collections.Generic;
namespace GenericLinkedListCollection
{
    public class Program
    {
        public static void Main()
        {
            Student student1 = new Student() { ID = 101, Name = "Anurag", Branch = "CSE" };
            Student student2 = new Student() { ID = 102, Name = "Mohanty", Branch = "CSE" };
            Student student3 = new Student() { ID = 103, Name = "Sambit", Branch = "CSE" };
            Student student4 = new Student() { ID = 104, Name = "Pranaya", Branch = "CSE" };

            LinkedList<Student> linkedList = new LinkedList<Student>();
            linkedList.AddLast(student1);
            linkedList.AddLast(student2);
            linkedList.AddLast(student3);
            linkedList.AddLast(student4);
            Console.WriteLine("LinkedList Elements");
            foreach (var item in linkedList)
            {
                Console.WriteLine($"Id: {item.ID}, Name: {item.Name}, Branch: {item.Branch}");
            }
            Console.ReadKey();
        }
    }

    public class Student
    {

```

```
        public int ID { get; set; }  
        public string Name { get; set; }  
        public string Branch { get; set; }  
    }  
}
```

Output:

```
LinkedList Elements  
Id: 101, Name: Anurag, Branch: CSE  
Id: 102, Name: Mohanty, Branch: CSE  
Id: 103, Name: Sambit, Branch: ETC  
Id: 104, Name: Pranaya, Branch: ETC
```

Advantages of Linked List in C#

1. They are dynamic in nature and allocate memory as and when required.
2. Insertion and deletion are easy to implement.
3. Other data structures such as Stack and Queue can also be implemented easily using Linked List.
4. It has faster access time and can be expanded in constant time without memory overhead.
5. Since there is no need to define an initial size for a linked list, hence memory utilization is effective.
6. Backtracking is possible in doubly-linked lists.

In the next article, I am going to discuss the [Concurrent Collections in C#](#) with Examples. Here, in this article, I try to explain the **Generic LinkedList<T> Collection Class in C#** with Examples. I hope this Generic LinkedList<T> Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.